NetApp儲存整合概述

NetApp Solutions

NetApp
May 17, 2022

This PDF was generated from https://docs.netapp.com/zh-tw/netapp-solutions/containers/anthos-with-netapp/a-w-n_trident_ontap_nfs.html on May 17, 2022. Always check docs.netapp.com for the latest.

目錄

NetApp儲存整合概述 1
Anthos Ready儲存合作夥伴方案。 1
NetApp儲存整合 1
Astra Trident總覽 1

NetApp儲存整合概述

Anthos Ready儲存合作夥伴方案。

Google Cloud會定期透過Anthos Ready儲存合作夥伴方案、要求更新合作夥伴儲存整合與新版Anthos的驗證。您可以找到目前已驗證的儲存解決方案、SCSI驅動程式、可用功能及所支援的Anthos版本清單 "請按這裡"。

NetApp每ONTAP 季都會定期遵循要求、驗證我們符合Astra Trident SCSI標準的儲存協調程式、以及我們的包含Anthos版本的更新版的更新版及元件儲存系統。

下表包含NetApp與NetApp合作夥伴工程師已測試的Anthos版本、以驗證NetApp Astra Trident SCSI驅動程式及功能集、作為Anthos Ready儲存合作夥伴方案的一部分：

部署類型	版本	儲存系統	Astra Trident版本	傳輸協定	功能
VMware	1.10	ONTAP	22.01	NAS	多寫入器、Volume Expansion、快照
VMware	1.10	ONTAP	22.01	SAN	原始區塊、Volume擴充、快照
VMware	1.10	元素	22.01	SAN	原始區塊、Volume擴充、快照
裸機	1.10	ONTAP	22.01	NAS	多寫入器、Volume Expansion、快照
裸機	1.10	ONTAP	22.01	SAN	原始區塊、Volume擴充、快照

NetApp儲存整合

NetApp提供多種產品來協助您協調及管理容器型環境（例如Anthos）中的持續資料。

NetApp Astra Trident是開放原始碼且完全支援的儲存協調工具、適用於容器和Kubernetes配送、包括Anthos。如需詳細資訊、請造訪Astra Trident網站 "請按這裡"。

下列頁面提供NetApp產品的其他資訊、這些產品已通過NetApp解決方案的Anthos應用程式與持續儲存管理驗證：

- "NetApp Astra Trident"

"下一步：進階組態選項：Anthos with NetApp。"

Astra Trident總覽

Astra Trident是開放原始碼且完全支援的儲存協調工具、適用於容器和Kubernetes配送、包括Anthos。Trident可

搭配整個NetApp儲存產品組合（包括NetApp ONTAP 的整套和Element儲存系統）使用、也支援NFS和iSCSI連線。Trident可讓終端使用者從NetApp儲存系統配置及管理儲存設備、而無需儲存管理員介入、進而加速DevOps工作流程。

系統管理員可根據專案需求和儲存系統模型來設定多個儲存後端、以啟用進階儲存功能、包括壓縮、特定磁碟類型或QoS層級、以保證特定層級的效能。定義後端後端之後、開發人員可在專案中使用這些後端來建立持續磁碟區宣告（PVCS）、並視需要將持續儲存附加至容器。

[錯誤：缺少圖形影像]

Astra Trident的開發週期很快、就像Kubernetes一樣、每年發行四次。

最新版的Astra Trident（22.01）於2022年1月發表。支援對照表、顯示哪些版本的Trident已通過測試、可找到Kubernetes經銷產品 "請按這裡"。

從20.04版本開始、Trident設定由Trident操作員執行。營運者讓大規模部署變得更簡單、並提供額外支援、包括在Trident安裝過程中部署的Pod自我修復。

有了21.01版、我們提供了Helm圖表、方便您安裝Trident操作員。

下載Astra Trident

若要在已部署的使用者叢集上安裝Trident並佈建持續磁碟區、請完成下列步驟：

1. 將安裝歸檔檔案下載至管理工作站、並擷取內容。目前的Trident版本為22.01、可下載 "請按這裡"。

```
[ubuntu@gke-admin-ws-2021-07-15 ~]$ wget
https://github.com/NetApp/trident/releases/download/v22.01.0/trident-
installer-22.01.0.tar.gz
--2021-05-06 15:17:30--
https://github.com/NetApp/trident/releases/download/v22.01.0/trident-
installer-22.01.0.tar.gz
Resolving github.com (github.com)... 140.82.114.3
Connecting to github.com (github.com)|140.82.114.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github-
releases.githubusercontent.com/77179634/a4fa9f00-a9f2-11eb-9053-
98e8e573d4ae?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-
Credential=AKIAIWNJYAX4CSVEH53A%2F20210506%2Fus-east-
1%2Fs3%2Faws4_request&X-Amz-Date=20210506T191643Z&X-Amz-Expires=300&X-
Amz-
Signature=8a49a2a1e08c147d1ddd8149ce45a5714f9853fee19bb1c507989b9543eb36
30&X-Amz-
SignedHeaders=host&actor_id=0&key_id=0&repo_id=77179634&response-
content-disposition=attachment%3B%20filename%3Dtrident-installer-
22.01.0.tar.gz&response-content-type=application%2Foctet-stream
[following]
--2021-05-06 15:17:30--  https://github-
releases.githubusercontent.com/77179634/a4fa9f00-a9f2-11eb-9053-
```

```
98e8e573d4ae?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-
Credential=AKIAIWNJYAX4CSVEH53A%2F20210506%2Fus-east-
1%2Fs3%2Faws4_request&X-Amz-Date=20210506T191643Z&X-Amz-Expires=300&X-
Amz-
Signature=8a49a2a1e08c147d1ddd8149ce45a5714f9853fee19bb1c507989b9543eb36
30&X-Amz-
SignedHeaders=host&actor_id=0&key_id=0&repo_id=77179634&response-
content-disposition=attachment%3B%20filename%3Dtrident-installer-
22.01.0.tar.gz&response-content-type=application%2Foctet-stream
Resolving github-releases.githubusercontent.com (github-
releases.githubusercontent.com)... 185.199.108.154, 185.199.109.154,
185.199.110.154, ...
Connecting to github-releases.githubusercontent.com (github-
releases.githubusercontent.com)|185.199.108.154|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 38349341 (37M) [application/octet-stream]
Saving to: 'trident-installer-22.01.0.tar.gz'

100%[=================================================================
==============================================>] 38,349,341  88.5MB/s
in 0.4s

2021-05-06 15:17:30 (88.5 MB/s) - 'trident-installer-22.01.0.tar.gz'
saved [38349341/38349341]
```

2. 從下載的套裝組合中擷取Trident安裝。

```
[ubuntu@gke-admin-ws-2021-07-15 ~]$ tar -xzf trident-installer-
22.01.0.tar.gz
[ubuntu@gke-admin-ws-2021-07-15 ~]$ cd trident-installer/
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$
```

使用Helm安裝Trident運算子

依照預設、不會在GKE管理工作站上安裝Helm。您可以使用Ubuntu中提供的APT工具輕鬆安裝。

1. 首先、將使用者叢集的「kubeconfig」檔案位置設定為環境變數、這樣您就不需要參考、因為Trident沒有傳遞此檔案的選項。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ export
KUBECONFIG=~/user-cluster-1/user-cluster-1-kubeconfig
```

2. 在使用者叢集中建立Trident命名空間時、請執行Helm命令、從Lm目錄的tar安裝Trident運算子。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ helm install trident
helm/trident-operator-22.01.0.tgz --create-namespace --namespace trident
NAME: trident
LAST DEPLOYED: Fri May  7 12:54:25 2021
NAMESPACE: trident
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
Thank you for installing trident-operator, which will deploy and manage
NetApp's Trident CSI
storage provisioner for Kubernetes.

Your release is named 'trident' and is installed into the 'trident'
namespace.
Please note that there must be only one instance of Trident (and
trident-operator) in a Kubernetes cluster.

To configure Trident to manage storage resources, you will need a copy
of tridentctl, which is
available in pre-packaged Trident releases.  You may find all Trident
releases and source code
online at https://github.com/NetApp/trident.

To learn more about the release, try:

  $ helm status trident
  $ helm get all trident
```

3. 您可以檢查在命名空間中執行的Pod、或使用tridentctl二進位檔檢查安裝的版本、以驗證Trident是否已成功安裝。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ oc get pods -n
trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-5z45l                   1/2     Running   2          30s
trident-csi-696b685cf8-htdb2        6/6     Running   0          30s
trident-csi-b74p2                   2/2     Running   0          30s
trident-csi-lrw4n                   2/2     Running   0          30s
trident-operator-7c748d957-gr2gw    1/1     Running   0          36s

[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ ./tridentctl -n
trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 22.01.0          | 22.01.0          |
+----------------+----------------+
```

在某些情況下、客戶環境可能需要自訂Trident部署。在這些情況下、您也可以手動安裝Trident運算子、並更新隨附的資訊清單、以自訂部署。

手動安裝Trident運算子

1. 首先、將使用者叢集的「kubeconfig」檔案位置設定為環境變數、這樣您就不需要參考、因為Trident沒有傳遞此檔案的選項。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ KUBECONFIG=~/user-
cluster-1/user-cluster-1-kubeconfig
```

2. 「Trident安裝程式」目錄包含定義所有必要資源的資訊清單。使用適當的資訊清單、建立「TridentOrchestrator」自訂資源定義。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
customresourcedefinition.apiextensions.k8s.io/tridentorchestrators.tride
nt.netapp.io created
```

3. 如果不存在、請使用提供的資訊清單、在叢集中建立Trident命名空間。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl apply -f
deploy/namespace.yaml
namespace/trident created
```

4. 建立Trident營運者部署所需的資源、例如營運者的「服務帳戶」、專屬的「PodSecurity Policy」、或營運者本身的「ClusterROLTE」和「ClusterROLESTBinding」。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
deploy/bundle.yaml
serviceaccount/trident-operator created
clusterrole.rbac.authorization.k8s.io/trident-operator created
clusterrolebinding.rbac.authorization.k8s.io/trident-operator created
deployment.apps/trident-operator created
podsecuritypolicy.policy/tridentoperatorpods created
```

5. 您可以使用下列命令來檢查部署後的操作員狀態：

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl get
deployment -n trident
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
trident-operator   1/1     1            1           23s
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl get pods -n
trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-operator-66f48895cc-lzczk   1/1     Running   0          41s
```

6. 部署營運者之後、我們就可以使用它來安裝Trident。這需要建立「TridentOrchestrator」。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
deploy/crds/tridentorchestrator_cr.yaml
tridentorchestrator.trident.netapp.io/trident created
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl describe
torc trident
Name:         trident
Namespace:
Labels:       <none>
Annotations:  <none>
API Version:  trident.netapp.io/v1
Kind:         TridentOrchestrator
Metadata:
  Creation Timestamp:  2021-05-07T17:00:28Z
  Generation:          1
  Managed Fields:
    API Version:  trident.netapp.io/v1
    Fields Type:  FieldsV1
    fieldsV1:
      f:spec:
        .:
```

```
        f:debug:
        f:namespace:
    Manager:      kubectl-create
    Operation:    Update
    Time:         2021-05-07T17:00:28Z
    API Version:  trident.netapp.io/v1
    Fields Type:  FieldsV1
    fieldsV1:
      f:status:
        .:
        f:currentInstallationParams:
          .:
          f:IPv6:
          f:autosupportHostname:
          f:autosupportImage:
          f:autosupportProxy:
          f:autosupportSerialNumber:
          f:debug:
          f:enableNodePrep:
          f:imagePullSecrets:
          f:imageRegistry:
          f:k8sTimeout:
          f:kubeletDir:
          f:logFormat:
          f:silenceAutosupport:
          f:tridentImage:
        f:message:
        f:namespace:
        f:status:
        f:version:
    Manager:         trident-operator
    Operation:       Update
    Time:            2021-05-07T17:00:28Z
  Resource Version:  931421
  Self Link:
/apis/trident.netapp.io/v1/tridentorchestrators/trident
  UID:               8a26a7a6-dde8-4d55-9b66-a7126754d81f
Spec:
  Debug:      true
  Namespace:  trident
Status:
  Current Installation Params:
    IPv6:                       false
    Autosupport Hostname:
    Autosupport Image:          netapp/trident-autosupport:21.01
    Autosupport Proxy:
```

```
    Autosupport Serial Number:
    Debug:                      true
    Enable Node Prep:           false
    Image Pull Secrets:
    Image Registry:
    k8sTimeout:           30
    Kubelet Dir:          /var/lib/kubelet
    Log Format:           text
    Silence Autosupport:  false
    Trident Image:        netapp/trident:22.01.0
  Message:                Trident installed
  Namespace:              trident
  Status:                 Installed
  Version:                v22.01.0
Events:
  Type    Reason      Age   From                        Message
  ----    ------      ----  ----                        -------
  Normal  Installing  80s   trident-operator.netapp.io  Installing
Trident
  Normal  Installed   68s   trident-operator.netapp.io  Trident
installed
```

7. 您可以檢查在命名空間中執行的Pod、或使用tridentctl二進位檔檢查安裝的版本、以驗證Trident是否已成功安裝。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl get pods -n
trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-bb64c6cb4-lmd6h         6/6     Running   0          82s
trident-csi-gn59q                   2/2     Running   0          82s
trident-csi-m4szj                   2/2     Running   0          82s
trident-csi-sb9k9                   2/2     Running   0          82s
trident-operator-66f48895cc-lzczk   1/1     Running   0          2m39s

[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ ./tridentctl -n
trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 22.01.0          | 22.01.0          |
+----------------+----------------+
```

建立儲存系統後端

完成Astra Trident操作員安裝之後、您必須為所使用的特定NetApp儲存平台設定後端。請依照下列連結繼續Astra Trident的設定與組態。

- "NetApp ONTAP 不適用於NFS"
- "NetApp ONTAP 支援iSCSI"
- "支援iSCSI NetApp Element"

"下一步：進階組態選項：Anthos with NetApp。"

NetApp ONTAP 不含NFS組態：Anthos with NetApp

若要與NetApp ONTAP 支援儲存系統進行Trident整合、您必須建立後端、以便與儲存系統進行通訊。

1. 下載的安裝歸檔文件中有「shame-INPUT」資料夾階層的範例後端檔案。對於ONTAP NetApp支援NFS的系統、請將「backender-ontap - nas.json」檔案複製到您的工作目錄、然後編輯檔案。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-
input/backends-samples/ontap-nas/backend-ontap-nas.json ./
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi backend-ontap-
nas.json
```

2. 編輯後端名稱、管理LIF、dataLIF、SVM、使用者名稱、 和密碼值。

```
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "ontap-nas+10.61.181.221",
    "managementLIF": "172.21.224.201",
    "dataLIF": "10.61.181.221",
    "svm": "trident_svm",
    "username": "cluster-admin",
    "password": "password"
}
```

> 最佳實務做法是將自訂的backendName值定義為storageDriverName和資料LIF的組合、以利NFS識別。

3. 在這個後端檔案就緒的情況下、執行下列命令來建立第一個後端。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ ./tridentctl -n
trident create backend -f backend-ontap-nas.json
+------------------------+----------------
+--------------------------------------+--------+---------+
|          NAME          | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+------------------------+----------------
+--------------------------------------+--------+---------+
| ontap-nas+10.61.181.221 | ontap-nas      | be7a619d-c81d-445c-b80c-
5c87a73c5b1e | online |       0 |
+------------------------+----------------
+--------------------------------------+--------+---------+
```

4. 建立後端之後、您必須接著建立儲存類別。就像後端一樣、範例輸入資料夾中也有可供編輯的儲存類別檔案範例。將其複製到工作目錄、並進行必要的編輯、以反映所建立的後端。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-
input/storage-class-samples/storage-class-csi.yaml.templ ./storage-
class-basic.yaml
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi storage-class-
basic.yaml
```

5. 唯一必須對此檔案進行的編輯、是從新建立的後端、將「backendType」值定義為儲存驅動程式名稱。另請注意名稱欄位值、此值必須在後續步驟中參考。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: basic-csi
provisioner: csi.trident.netapp.io
parameters:
  backendType: "ontap-nas"
```

> 在此檔案中定義了一個名為「FSType」的選用欄位。此行可在NFS後端刪除。

6. 執行「kubecll」命令來建立儲存類別。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
storage-class-basic.yaml
storageclass.storage.k8s.io/basic-csi created
```

7. 建立儲存類別之後、您必須建立第一個持續磁碟區宣告（PVc）。還有一個「PVC-base.yaml」檔案範例、

也可在範例輸入中執行此動作。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-input/pvc-
samples/pvc-basic.yaml ./
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi pvc-basic.yaml
```

8. 唯一必須對此檔案進行的編輯、是確保「儲存類別名稱」欄位符合剛剛建立的欄位。您可以根據要配置的工作負載需求、進一步自訂PVc定義。

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: basic
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: basic-csi
```

9. 使用「kubecll」命令建立永久虛電路。視所建立的備用磁碟區大小而定、建立作業可能需要一些時間、因此您可以在完成時觀看程序。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
pvc-basic.yaml
persistentvolumeclaim/basic created

[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl get pvc
NAME     STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
basic    Bound    pvc-b4370d37-0fa4-4c17-bd86-94f96c94b42d   1Gi
RWO            basic-csi      7s
```

"下一步：進階組態選項：Anthos with NetApp。"

NetApp ONTAP 支援iSCSI組態：Anthos with NetApp

若要與NetApp ONTAP 支援儲存系統進行Trident整合、您必須建立後端、以便與儲存系統進行通訊。

1. 下載的安裝歸檔文件中有「shame-INPUT」資料夾階層的範例後端檔案。對於ONTAP 供應iSCSI的NetApp支援系統、請將「backender-ontap - san・json」檔案複製到您的工作目錄、然後編輯該檔案。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-
input/backends-samples/ontap-san/backend-ontap-san.json ./
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi backend-ontap-
san.json
```

2. 編輯此檔案中的管理LIF、dataLIF、SVM、使用者名稱和密碼值。

```
{
  "version": 1,
  "storageDriverName": "ontap-san",
  "managementLIF": "172.21.224.201",
  "dataLIF": "10.61.181.240",
  "svm": "trident_svm",
  "username": "admin",
  "password": "password"
}
```

3. 在這個後端檔案就緒的情況下、執行下列命令來建立第一個後端。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ ./tridentctl -n
trident create backend -f backend-ontap-san.json
+------------------------+----------------
+--------------------------------------+--------+---------+
|          NAME          | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+------------------------+----------------
+--------------------------------------+--------+---------+
| ontapsan_10.61.181.241 | ontap-san      | 6788533c-7fea-4a35-b797-
fb9bb3322b91 | online |       0 |
+------------------------+----------------
+--------------------------------------+--------+---------+
```

4. 建立後端之後、您必須接著建立儲存類別。就像後端一樣、範例輸入資料夾中也有可供編輯的儲存類別檔案範例。將其複製到工作目錄、並進行必要的編輯、以反映所建立的後端。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-
input/storage-class-samples/storage-class-csi.yaml.templ ./storage-
class-basic.yaml
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi storage-class-
basic.yaml
```

5. 唯一必須對此檔案進行的編輯、是從新建立的後端、將「backendType」值定義為儲存驅動程式名稱。另請

注意名稱欄位值、此值必須在後續步驟中參考。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: basic-csi
provisioner: csi.trident.netapp.io
parameters:
  backendType: "ontap-san"
```

> 在此檔案中定義了一個名為「FSType」的選用欄位。在iSCSI後端中、此值可設定為特定的Linux檔案系統類型（XFS、ext4等）、也可刪除以允許OpenShift決定要使用的檔案系統。

6. 執行「kubecll」命令來建立儲存類別。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
storage-class-basic.yaml
storageclass.storage.k8s.io/basic-csi created
```

7. 建立儲存類別之後、您必須建立第一個持續磁碟區宣告（PVc）。還有一個「PVC-base.yaml」檔案範例、也可在範例輸入中執行此動作。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-input/pvc-
samples/pvc-basic.yaml ./
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi pvc-basic.yaml
```

8. 唯一必須對此檔案進行的編輯、是確保「儲存類別名稱」欄位符合剛剛建立的欄位。您可以根據要配置的工作負載需求、進一步自訂PVc定義。

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: basic
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: basic-csi
```

9. 使用「kubecll」命令建立永久虛電路。視所建立的備用磁碟區大小而定、建立作業可能需要一些時間、因此您可以在完成時觀看程序。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
pvc-basic.yaml
persistentvolumeclaim/basic created

[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl get pvc
NAME     STATUS    VOLUME                                     CAPACITY
ACCESS MODES    STORAGECLASS    AGE
basic    Bound     pvc-7ceac1ba-0189-43c7-8f98-094719f7956c   1Gi
RWO            basic-csi      3s
```

"下一步：進階組態選項：Anthos with NetApp。"

支援NetApp的Anthos iSCSI組態NetApp Element

若要啟用Trident與NetApp Element 支援功能的整合、您必須建立後端、以便使用iSCSI傳輸協定與儲存系統進行通訊。

1. 下載的安裝歸檔文件中有「shame-INPUT」資料夾階層的範例後端檔案。若NetApp Element 為供應iSCSI的支援系統、請將「backend-solidfire.json」檔案複製到您的工作目錄、然後編輯檔案。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-
input/backends-samples/solidfire/backend-solidfire.json ./
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi ./backend-
solidfire.json
```

   a. 編輯「端點」行上的使用者、密碼和MVIP值。
   b. 編輯「VIP」值。

```
 {
    "version": 1,
    "storageDriverName": "solidfire-san",
    "Endpoint": "https://trident:password@172.21.224.150/json-
rpc/8.0",
    "SVIP": "10.61.180.200:3260",
    "TenantName": "trident",
    "Types": [{"Type": "Bronze", "Qos": {"minIOPS": 1000, "maxIOPS":
2000, "burstIOPS": 4000}},
              {"Type": "Silver", "Qos": {"minIOPS": 4000, "maxIOPS":
6000, "burstIOPS": 8000}},
              {"Type": "Gold", "Qos": {"minIOPS": 6000, "maxIOPS":
8000, "burstIOPS": 10000}}]
}
```

2. 在這個後端檔案就位的情況下、執行下列命令來建立您的第一個後端。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ ./tridentctl -n
trident create backend -f backend-solidfire.json
+------------------------+----------------
+--------------------------------------+--------+---------+
|          NAME          | STORAGE DRIVER |                  UUID
| STATE  | VOLUMES |
+------------------------+----------------
+--------------------------------------+--------+---------+
| solidfire_10.61.180.200 | solidfire-san  | b90783ee-e0c9-49af-8d26-
3ea87ce2efdf | online |       0 |
+------------------------+----------------
+--------------------------------------+--------+---------+
```

3. 建立後端之後、您必須接著建立儲存類別。就像後端一樣、範例輸入資料夾中也有可供編輯的儲存類別檔案範例。將其複製到工作目錄、並進行必要的編輯、以反映所建立的後端。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-
input/storage-class-samples/storage-class-csi.yaml.templ ./storage-
class-basic.yaml
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi storage-class-
basic.yaml
```

4. 唯一必須對此檔案進行的編輯、是從新建立的後端、將「backendType」值定義為儲存驅動程式名稱。另請注意名稱欄位值、此值必須在後續步驟中參考。

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: basic-csi
provisioner: csi.trident.netapp.io
parameters:
  backendType: "solidfire-san"
```

> 在此檔案中定義了一個名為「FSType」的選用欄位。在iSCSI後端中、此值可設定為特定的Linux檔案系統類型（XFS、ext4等）、也可刪除此值、讓OpenShift決定要使用的檔案系統。

5. 執行「kubecll」命令來建立儲存類別。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
storage-class-basic.yaml
storageclass.storage.k8s.io/basic-csi created
```

6. 建立儲存類別之後、您必須建立第一個持續磁碟區宣告（PVc）。還有一個「PVC-base.yaml」檔案範例、也可在範例輸入中執行此動作。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ cp sample-input/pvc-
samples/pvc-basic.yaml ./
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ vi pvc-basic.yaml
```

7. 唯一必須對此檔案進行的編輯、是確保「儲存類別名稱」欄位符合剛剛建立的欄位。您可以根據要配置的工作負載需求、進一步自訂PVc定義。

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: basic
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: basic-csi
```

8. 使用「kubecll」命令建立永久虛電路。視所建立的備用磁碟區大小而定、建立作業可能需要一些時間、因此您可以在完成時觀看程序。

```
[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl create -f
pvc-basic.yaml
persistentvolumeclaim/basic created

[ubuntu@gke-admin-ws-2021-07-15 trident-installer]$ kubectl get pvc
NAME     STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   AGE
basic    Bound    pvc-3445b5cc-df24-453d-a1e6-b484e874349d   1Gi
RWO            basic-csi      5s
```

"下一步：進階組態選項：Anthos with NetApp。"

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.